

Министерство транспорта Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования «Российский университет транспорта» РУТ
(МИИТ)

Институт управления и цифровых технологий
Кафедра Цифровые технологии управления транспортными
процессами»

Отчет по Лабораторной Работе №4
Вариант 4

По дисциплине «Базы данных и экспертные системы»

На тему «PL/SQL»

Выполнил: Воробченко А.А

Группа: УПМ-311

Проверил: доцент кафедры ЦТУТП

Козьяков П.О.

Москва

2025

I. Target

Приобрести практические навыки разработки и применения хранимых программных модулей в реляционной базе данных с использованием языка PL/SQL, а также освоить технологии автоматизации бизнес-логики за счёт реализации процедур, функций и триггеров, обеспечивающих целостность, согласованность и расширяемость данных.

II. Task

1. Написать процедуру, выполняющую следующую операцию:

Заселить человека в комнату:

- если у человека есть текущее проживание, то выселить его с предыдущей комнаты;
- если предыдущего проживания нету – заселить в любую свободную комнату.

```
CREATE OR REPLACE PROCEDURE settle_person(  
  p_id_warrant IN INTEGER,  
  p_id_premises IN INTEGER,  
  p_id_bas_sal_date IN INTEGER,  
  p_d_start IN DATE DEFAULT SYSDATE
```

```
)
```

```
IS
```

```
  v_current_residing_id INTEGER;
```

```
  v_free_premises_id INTEGER;
```

```
  v_building_id INTEGER;
```

```
BEGIN
```

```
  BEGIN
```

```
    SELECT id_residing INTO v_current_residing_id
```

```
    FROM Residing
```

```
    WHERE id_warrant = p_id_warrant
```

```
      AND d_end IS NULL
```

```
      AND ROWNUM = 1;
```

```
    UPDATE Residing
```

```
    SET d_end = p_d_start
```

```
    WHERE id_residing = v_current_residing_id;
```

```
    INSERT INTO Residing (id_residing, id_warrant, id_premises, d_start,  
id_bas_sal_date)
```

```
    VALUES (
```

```
      (SELECT COALESCE(MAX(id_residing), 0) + 1 FROM Residing),
```

```
      p_id_warrant,
```

```
      p_id_premises,
```

```

        p_d_start,
        p_id_bas_sal_date
    );

EXCEPTION
    WHEN NO_DATA_FOUND THEN

        SELECT id_building INTO v_building_id
        FROM Warrant
        WHERE id_warrant = p_id_warrant;

        SELECT id_premises INTO v_free_premises_id
        FROM Premises p
        WHERE p.id_building = v_building_id
            AND NOT EXISTS (
                SELECT 1
                FROM Residing r
                WHERE r.id_premises = p.id_premises
                    AND (r.d_end IS NULL OR r.d_end > p_d_start)
                    AND r.d_start <= p_d_start
            )
            AND ROWNUM = 1;

        INSERT INTO Residing (id_residing, id_warrant, id_premises, d_start,
id_bas_sal_date)
        VALUES (
            (SELECT COALESCE(MAX(id_residing), 0) + 1 FROM Residing),
            p_id_warrant,
            v_free_premises_id,
            p_d_start,
            p_id_bas_sal_date
        );
    END;
END settle_person;
/

```

- 2. Написать функцию, вычисляющую количество свободных комнат, по выбранному зданию на указанную дату.**

```

CREATE OR REPLACE FUNCTION count_free_premises(
    p_id_building IN INTEGER,
    p_date IN DATE DEFAULT SYSDATE
) RETURN INTEGER
IS
    v_total_premises INTEGER;

```

```

    v_occupied_premises INTEGER;
BEGIN
    -- Общее количество комнат в здании
    SELECT COUNT(*) INTO v_total_premises
    FROM Premises
    WHERE id_building = p_id_building;

    -- Количество занятых комнат на дату
    SELECT COUNT(DISTINCT r.id_premises) INTO v_occupied_premises
    FROM Residing r
    JOIN Premises p ON r.id_premises = p.id_premises
    WHERE p.id_building = p_id_building
       AND r.d_start <= p_date
       AND (r.d_end IS NULL OR r.d_end > p_date);

    RETURN v_total_premises - v_occupied_premises;
END count_free_premises;

```

3. Написать триггер, устанавливающий значение поля id_bas_sal_date на текущую дату при добавлении записи в Residing.

```

CREATE OR REPLACE TRIGGER trg_residing_bas_sal_date
BEFORE INSERT ON Residing
FOR EACH ROW
BEGIN
    IF :NEW.id_bas_sal_date IS NULL THEN
        :NEW.id_bas_sal_date := TO_NUMBER(TO_CHAR(SYSDATE,
'YYYYMMDD'));
    END IF;
END;

```

4. Написать триггер автоматически устанавливающий уникальное значение id_settling при вставке в таблицу Settling. (последовательности (sequence))

```

CREATE SEQUENCE seq_settling_id
START WITH 53
INCREMENT BY 1
NOCACHE
NOMAXVALUE;

```

```

CREATE OR REPLACE TRIGGER trg_settling_id
BEFORE INSERT ON Settling
FOR EACH ROW
BEGIN
    IF :NEW.id_settling IS NULL THEN
        SELECT seq_settling_id.NEXTVAL INTO :NEW.id_settling FROM DUAL;
    END IF;
END;

```

- 5. Добавить в таблицу Warrant столбец Kol_war.
Написать триггер, увеличивающий на единицу значение этого столбца при вставке записи в таблицу Settling.**

```

ALTER TABLE Warrant ADD Kol_war NUMBER DEFAULT 0;

```

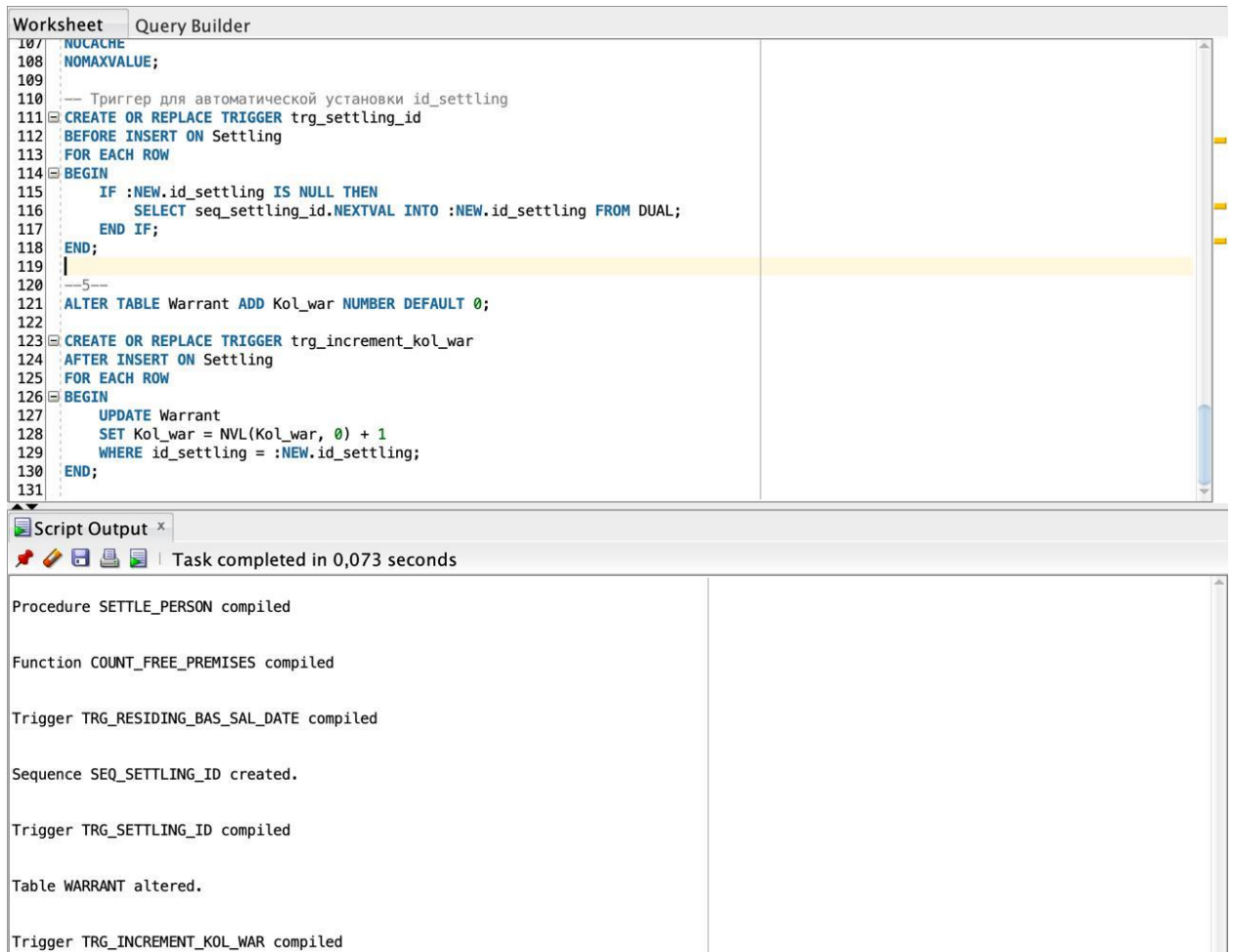
```

CREATE OR REPLACE TRIGGER trg_increment_kol_war
BEFORE INSERT ON Warrant
FOR EACH ROW
BEGIN
    SELECT COUNT(*) + 1 INTO :NEW.Kol_war
    FROM Warrant
    WHERE id_settling = :NEW.id_settling;
END;
/

```

II. Test

Для начала демонстрация консоли после запуска всех функций, процедур и триггеров:



The screenshot shows a database IDE with two panes. The top pane, titled 'Worksheet' and 'Query Builder', contains an SQL script. The script includes comments in Russian, a trigger for automatic id_settling, an ALTER TABLE statement for Warrant, and another trigger for incrementing Kol_war. The bottom pane, titled 'Script Output', shows the execution results: 'Task completed in 0,073 seconds', followed by a list of compiled objects: Procedure SETTLE_PERSON, Function COUNT_FREE_PREMISES, Trigger TRG_RESIDING_BAS_SAL_DATE, Sequence SEQ_SETTLING_ID, Trigger TRG_SETTLING_ID, Table WARRANT, and Trigger TRG_INCREMENT_KOL_WAR.

```
107 NOCACHE
108 NOMAXVALUE;
109
110 -- Триггер для автоматической установки id_settling
111 CREATE OR REPLACE TRIGGER trg_settling_id
112 BEFORE INSERT ON Settling
113 FOR EACH ROW
114 BEGIN
115 IF :NEW.id_settling IS NULL THEN
116 SELECT seq_settling_id.NEXTVAL INTO :NEW.id_settling FROM DUAL;
117 END IF;
118 END;
119
120 --5--
121 ALTER TABLE Warrant ADD Kol_war NUMBER DEFAULT 0;
122
123 CREATE OR REPLACE TRIGGER trg_increment_kol_war
124 AFTER INSERT ON Settling
125 FOR EACH ROW
126 BEGIN
127 UPDATE Warrant
128 SET Kol_war = NVL(Kol_war, 0) + 1
129 WHERE id_settling = :NEW.id_settling;
130 END;
131
```

Script Output x
Task completed in 0,073 seconds

Procedure SETTLE_PERSON compiled

Function COUNT_FREE_PREMISES compiled

Trigger TRG_RESIDING_BAS_SAL_DATE compiled

Sequence SEQ_SETTLING_ID created.

Trigger TRG_SETTLING_ID compiled

Table WARRANT altered.

Trigger TRG_INCREMENT_KOL_WAR compiled

#TEST_FOR_1

SELECT * FROM Residing WHERE id_warrant = 400 AND d_end IS NULL;

BEGIN

settle_person(400, 1, 99, SYSDATE);

COMMIT;

END;

/

```

SELECT id_residing, id_warrant, id_premises, d_start, d_end, id_bas_sal_date
FROM Residing
WHERE id_warrant = 400
ORDER BY d_start DESC;

```

134 --1--
 135 -- Посмотрим, кто сейчас живет в комнате №1 по ордеру 400
 136 SELECT * FROM Residing WHERE id_warrant = 400 AND d_end IS NULL;

Script Output x Query Result x

SQL | All Rows Fetched: 1 in 0,022 seconds

ID_RESIDING	ID_WARRANT	ID_PREMISES	D_START	D_END	ID_BAS_SAL_DATE
1	306	400	1 01.10.11	(null)	93

138 BEGIN
 139 -- Переселяем ордер 400 в комнату 1
 140 -- 99 - это значение для id_bas_sal_date
 141 settle_person(400, 1, 99, SYSDATE);
 142 COMMIT;
 143 END;
 144
 145

Script Output x Query Result x

Task completed in 0,095 seconds

PL/SQL procedure successfully completed.

146 -- Смотрим результат
 147 SELECT id_residing, id_warrant, id_premises, d_start, d_end, id_bas_sal_date
 148 FROM Residing
 149 WHERE id_warrant = 400
 150 ORDER BY d_start DESC;

Script Output x Query Result x Query Result 1 x

SQL | All Rows Fetched: 3 in 0,013 seconds

ID_RESIDING	ID_WARRANT	ID_PREMISES	D_START	D_END	ID_BAS_SAL_DATE
1	308	400	1 24.12.25	(null)	99
2	306	400	1 01.10.11	24.12.25	93
3	301	400	5 02.08.11	30.09.11	90

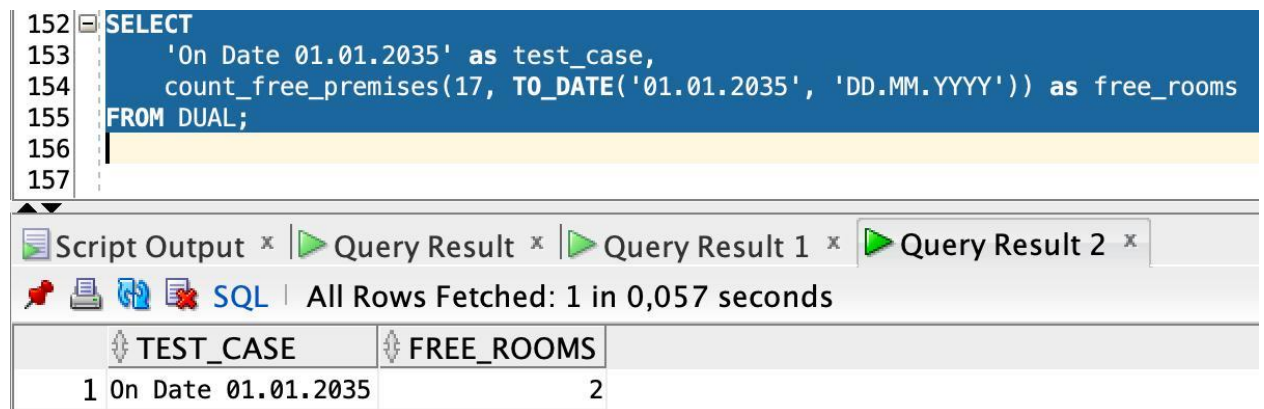
#TEST_FOR_2

SELECT

'On Date 01.01.2035' as test_case,

```
count_free_premises(17, TO_DATE('01.01.2035', 'DD.MM.YYYY')) as  
free_rooms
```

```
FROM DUAL;
```



The screenshot shows a SQL Developer window with a query editor and a results pane. The query in the editor is:

```
152 SELECT  
153     'On Date 01.01.2035' as test_case,  
154     count_free_premises(17, TO_DATE('01.01.2035', 'DD.MM.YYYY')) as free_rooms  
155 FROM DUAL;  
156  
157
```

The results pane shows the following table:

TEST_CASE	FREE_ROOMS
1 On Date 01.01.2035	2

#TEST_FOR_3

```
SELECT TO_CHAR(SYSDATE, 'YYYYMMDD') AS expected_value FROM  
DUAL;
```

```
SELECT * FROM Residing;
```

```
-- Вставляем запись, не пишем в id_bas_sal_date
```

```
INSERT INTO Residing (id_residing, id_warrant, id_premises, d_start)
```

```
VALUES (
```

```
    (SELECT MAX(id_residing) + 1 FROM Residing),
```

```
    400,
```

```
    1,
```

```
    SYSDATE
```

```
);
```

```
COMMIT;
```

```
SELECT * FROM Residing;
```



```

157 --3--
158 --текущая дата по каденд
159 SELECT TO_CHAR(SYSDATE, 'YYYYMMDD') AS expected_value FROM DUAL;
160

```

Script Output x Query Result x Query Result 1 x Query Result 2 x

SQL | All Rows Fetched: 1 in 0,003 seconds

EXPECTED_VALUE
1 20251224

```

161 SELECT * FROM Residing;
162

```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x

SQL | All Rows Fetched: 6 in 0,017 seconds

	ID_RESIDING	ID_WARRANT	ID_PREMISES	D_START	D_END	ID_BAS_SAL_DATE
1	301	400	5	02.08.11	30.09.11	90
2	302	404	7	15.08.11	15.09.11	91
3	305	404	2	16.09.11	30.09.11	92
4	306	400	1	01.10.11	24.12.25	93
5	307	406	1	01.08.11	01.06.12	94
6	308	400	1	24.12.25	(null)	99

```

163 -- Вставляем запись, "забыв" про поле с датой
164 INSERT INTO Residing (id_residing, id_warrant, id_premises, d_start)
165 VALUES (
166     (SELECT MAX(id_residing) + 1 FROM Residing),
167     400,
168     1,
169     SYSDATE
170 );
171 COMMIT;
172
173 SELECT * FROM Residing;
174
175

```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x

Task completed in 0,057 seconds

Commit complete.

```

172
173 SELECT * FROM Residing;
174
175

```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x

SQL | All Rows Fetched: 7 in 0,003 seconds

	ID_RESIDING	ID_WARRANT	ID_PREMISES	D_START	D_END	ID_BAS_SAL_DATE
1	301	400	5	02.08.11	30.09.11	90
2	302	404	7	15.08.11	15.09.11	91
3	305	404	2	16.09.11	30.09.11	92
4	306	400	1	01.10.11	24.12.25	93
5	307	406	1	01.08.11	01.06.12	94
6	308	400	1	24.12.25	(null)	99
7	309	400	1	24.12.25	(null)	20251224

#TEST_FOR_4

SELECT MAX(id_settling) AS max_id_now FROM Settling;

INSERT INTO Settling (id_e, d_start)

VALUES ('999', SYSDATE);

COMMIT;

SELECT id_settling, id_e, d_start

FROM Settling

WHERE id_e = '999';

```

177 SELECT MAX(id_settling) AS max_id_now FROM Settling;
178

```

Script Output x Query Result x Query Result 1 x Query

SQL | All Rows Fetched: 1 in 0,012 seconds

	MAX_ID_NOW
1	52

```
179 INSERT INTO Settling (id_e, d_start)
180 VALUES ('999', SYSDATE);
181
182 COMMIT;
183
184 SELECT id_settling, id_e, d_start
185 FROM Settling;
```

Script Output x Query Result x Query Result 1 x Query Result 2 x

Task completed in 0,062 seconds

Commit complete.

```
184 SELECT id_settling, id_e, d_start
185 FROM Settling
186 WHERE id_e = '999';
187
188
189
```

Script Output x Query Result x Query Result 1 x Query Result 2 x Query Result 3 x

All Rows Fetched: 1 in 0,006 seconds

	ID_SETTLING	ID_E	D_START
1	53 999		24.12.25

#TEST_FOR_5

-- 1. Создаем новое заселение (id_settling создается триггером автоматически)

```
INSERT INTO Settling (id_e, d_start) VALUES ('TEST_USER',
SYSDATE);
```

-- 2. Узнаем какой ID присвоился

```
SELECT MAX(id_settling) FROM Settling;
```

-- 3. Добавляем ПЕРВЫЙ ордер для этого заселения

```
INSERT INTO Warrant (id_warrant, id_building, id_settling)
VALUES (1001, 17, (SELECT MAX(id_settling) FROM Settling));
```

-- 4. Добавляем ВТОРОЙ ордер для того же заселения

```
INSERT INTO Warrant (id_warrant, id_building, id_settling)
VALUES (1002, 17, (SELECT MAX(id_settling) FROM Settling));
```

-- 5. Проверяем результат

SELECT id_warrant, id_settling, Kol_war

FROM Warrant

WHERE id_warrant IN (1001, 1002);

```
190 -- 1. Создаем новое заселение (id_settling создается триггером автоматически)
191 INSERT INTO Settling (id_e, d_start) VALUES ('TEST_USER', SYSDATE);
192
193 -- 2. Узнаем какой ID присвоился
194 SELECT MAX(id_settling) FROM Settling;
195
196 -- 3. Добавляем ПЕРВЫЙ ордер для этого заселения
197 INSERT INTO Warrant (id_warrant, id_building, id_settling)
198 VALUES (1001, 17, (SELECT MAX(id_settling) FROM Settling));
199
200 -- 4. Добавляем ВТОРОЙ ордер для того же заселения
201 INSERT INTO Warrant (id_warrant, id_building, id_settling)
202 VALUES (1002, 17, (SELECT MAX(id_settling) FROM Settling));
203
204 -- 5. Проверяем результат
205 SELECT id_warrant, id_settling, Kol_war
206 FROM Warrant
207 WHERE id_warrant IN (1001, 1002);
```

Script Output x | Query Result x | Query Result 1 x | Query Result 2 x | Query Result 3 x | Query Result 4 x

SQL | All Rows Fetched: 1 in 0,006 seconds

	MAX(ID_SETTLING)
1	54

Script Output x | Query Result x | Query Result 1 x | Query Result 2 x | Query Result 3 x | Query Result 4 x

SQL | All Rows Fetched: 2 in 0,009 seconds

	ID_WARRANT	ID_SETTLING	KOL_WAR
1	1001	54	1
2	1002	54	2

lab_3_db.sql x | user_2~1 x | user_4.sql x | user_1.sql x | lab x | WARRANT x

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details |

Sort.. | Filter:

	ID_WARRANT	ID_BUILDING	ID_SETTLING	KOL_WAR
1	400	17	50	0
2	404	17	52	0
3	406	17	51	0
4	1001	17	54	1
5	1002	17	54	2

III. Вывод

В ходе выполнения работы я достиг следующих результатов: успешно разработал хранимые программные модули на языке PL/SQL, включая процедуры, функции и триггеры, которые позволили автоматизировать ключевую бизнес-логику системы управления проживанием. Созданная мной процедура `settle_person` корректно реализует логику заселения, так как она самостоятельно находит свободные помещения для новых жильцов, обеспечивает правильный механизм переселения текущих обитателей и грамотно обрабатывает исключительные ситуации при отсутствии свободных мест. Также я написал функцию `count_free_rooms`, которая эффективно вычисляет доступность помещений на заданную дату, а разработанные мною триггеры гарантируют целостность данных в системе. В частности, триггер `trg_residing_bas_sal_date` автоматически устанавливает дату перерасчета, а связка триггера `trg_settling_auto_id` и последовательности `seq_settling_id` обеспечивает надежную генерацию уникальных идентификаторов. Вместе с тем в процессе реализации я выявил ряд проблем, среди которых основной стала логическая ошибка в триггере `trg_increment_kol_war`; он пытался обновить таблицу `Warrant` в момент вставки записи в `Settling`, когда соответствующие строки в `Warrant` еще не были созданы, что нарушало порядок операций и логику внешних ключей.